

Cyber Security Report at Code Alpha

A Project Report

submitted in partial fulfilment of the requirements for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering

Submitted by

Divyansh Sikarwar

(CS-23411624)

Under the supervision of

Mr. Rakesh Pandey

Assistant Professor



2

IILM University, Greater Noida, Uttar Pradesh

(November 2025)

Abstract

The Cyber Security Internship at Code Alpha provided comprehensive hands-on exposure to essential aspects of modern cybersecurity, encompassing areas such as network analysis, secure coding, intrusion detection, and phishing awareness. This program emphasized both theoretical understanding and practical implementation of techniques critical to safeguarding digital infrastructures.

Throughout the internship, various tasks were undertaken, including the development of a Network Sniffer to capture and analyse real-time data packets, a Phishing Awareness Training Module to educate users about social engineering attacks, and a Secure Coding Review to identify and mitigate vulnerabilities within software applications. Additionally, a Network Intrusion Detection System (NIDS) was configured to detect and respond to potential threats using open-source tools such as *Snort* and *Suricata*.

The internship not only strengthened technical competencies in network security, ethical hacking, and threat detection, but also enhanced critical thinking, problem-solving, and documentation skills. Under expert mentorship, the program bridged the gap between academic learning and real-world cybersecurity practices. The results of this internship highlight the importance of proactive defence mechanisms, awareness training, and secure software development in maintaining resilient digital systems

Keywords: Network Sniffer, Intrusion Detection System (IDS), Secure Coding, Phishing Awareness, Ethical Hacking, Threat Detection.



Student ID: CA/JU1/22512

CERTIFICATE OF COMPLETION

This Certificate Is Proudly Presented To

Divyansh Sikarwar

Was an active Participant at CodeAlpha Virtual Internship Program in Cyber Security
Internship with dedication and hard work. We really appreciate your efforts taken and
wish you all the best for the further.

1st July 2025 to 30th July 2025

Swati Sri

CEO & Founder

1st August 2025

Date of Issue



Contents

Abstract	0
1 Introduction	4
1.1 Overview	4
1.2 Objectives	4
1.3 Problem Statement	5
2 Literature Review	6
3 Methodology	8
3.1 Architecture Overview	8
3.2 Hardware Components	9
3.3 Software Tools	9
4 Implementation	11
5 Results and Discussion	15
6 Conclusion and Future Scope	19
6.1 Conclusion	19
6.2 Future Scope	19
References	20

1. Introduction

1.1 Overview

The rapid advancement of technology and digital transformation has led to a corresponding rise in cyber threats that target individuals, organizations, and governments alike. As businesses increasingly rely on interconnected networks and cloud infrastructures, the demand for robust cybersecurity measures has become more critical than ever. The Cyber Security Internship Program at CodeAlpha was designed to provide students with a comprehensive understanding of real-world cybersecurity challenges and to equip them with the necessary skills to combat such threats effectively.

Through a structured combination of theory and practical implementation, this internship offered hands-on exposure to essential domains of cybersecurity such as network traffic analysis, phishing prevention, secure coding practices, and intrusion detection systems. The program focused on developing technical proficiency using tools like Scapy, Snort, and Suricata, along with an emphasis on ethical hacking methodologies and risk assessment techniques. Interns were guided by industry professionals to enhance their analytical thinking, problem-solving abilities, and awareness of modern security practices.

1.2 Objectives

The primary objectives of this internship were:

1. To gain practical exposure to cybersecurity concepts, tools, and techniques used in the protection of digital assets.
2. To understand and analyse **network traffic** through the implementation of a **Network Sniffer** using Python.
3. To create a **Phishing Awareness Training Module** aimed at educating users about identifying and preventing phishing attacks.
4. To perform a **Secure Coding Review** to identify vulnerabilities and recommend remediation strategies for safer code development.
5. To design and implement a **Network Intrusion Detection System (NIDS)** for detecting suspicious network activity.
6. To strengthen knowledge in ethical hacking, data protection, and system security through real-world case studies.
7. To develop technical documentation and reporting skills aligned with industry standards.

1.3 Problem Statement

In the current digital era, cyberattacks are becoming increasingly sophisticated, posing significant risks to personal data, organizational systems, and national infrastructures. Many individuals and institutions lack the awareness and technical capability to identify or respond to these threats effectively. Common problems include **phishing attacks**, **unsecured coding practices**, **weak network monitoring**, and **delayed detection of intrusions**.

This internship was initiated to address these challenges by providing a learning platform where students could **analyse, design, and implement practical cybersecurity solutions**. Through hands-on projects, such as creating a network sniffer and intrusion detection system, the internship aimed to bridge the gap between theoretical knowledge and its real-world application, preparing participants to contribute meaningfully to the cybersecurity domain.

2. Literature Review

The field of Cyber Security has evolved significantly over the past few decades, driven by the exponential growth of digital technologies and the increasing sophistication of cyber threats. Researchers and industry experts have continuously proposed and implemented various strategies to secure networks, systems, and data from malicious actors. This section reviews key concepts, tools, and prior works relevant to the internship tasks undertaken—namely Network Sniffing, Phishing Awareness, Secure Coding Practices, and Intrusion Detection Systems.

2.1 Network Sniffing and Traffic Analysis

Network sniffing, also referred to as packet capturing or network monitoring, plays a crucial role in identifying network performance issues and detecting suspicious activities. According to Stallings (2017), analyzing packet-level data enables security analysts to monitor traffic flow, detect intrusions, and troubleshoot communication anomalies. Tools such as Wireshark, TCP dump, and Scapy have become standard in network forensics, providing insights into source and destination IP addresses, port numbers, and payload content.

In this internship, a Python-based Network Sniffer was developed using libraries like Scapy to capture and interpret network packets in real time. This approach aligns with the methods suggested by Bhattacharya & Islam (2020), emphasizing the role of open-source tools in learning and applying network security analysis techniques.

2.2 Phishing Attacks and Awareness Training

Phishing remains one of the most prevalent forms of social engineering attacks. As highlighted by Jagatic et al. (2007), phishing exploits human psychology rather than technical vulnerabilities, making user awareness a vital line of defense. Training programs and awareness campaigns significantly reduce susceptibility to such attacks.

Recent studies by Gupta & Singhal (2021) demonstrate that interactive training modules—which include real-world examples, visual cues, and quizzes—improve user vigilance against fake websites and deceptive emails. The Phishing Awareness Training Module developed during this internship incorporated these pedagogical elements, reinforcing concepts such as email verification, URL inspection, and identifying fraudulent requests.

2.3 Secure Coding Practices

Software vulnerabilities often arise due to insecure coding techniques, inadequate validation, and improper error handling. According to OWASP (Open Web Application Security Project), the top critical vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), and Buffer Overflow stem from coding flaws.

Research by McGraw (2013) emphasizes the necessity of integrating security into every stage of the software development lifecycle (SDLC), from design to deployment. The Secure Coding Review conducted in this internship followed OWASP guidelines to audit sample codebases, identify weaknesses, and recommend mitigations like input sanitization, proper authentication mechanisms, and secure API usage. This practical review bridged academic knowledge with professional security auditing practices.

2.4 Intrusion Detection Systems (IDS)

An Intrusion Detection System (IDS) monitors network or system activities for malicious behaviour or policy violations. Denning (1987) introduced one of the first models for anomaly detection, which became the foundation for modern IDS mechanisms. There are primarily two types: Host-Based IDS (HIDS) and Network-Based IDS (NIDS).

Contemporary tools like Snort, Suricata, and Zeek (formerly Bro) utilize signature-based and anomaly-based detection methods to identify potential intrusions. Research by García-Teodoro et al. (2009) categorizes IDS techniques into statistical analysis, machine learning, and knowledge-based approaches. During this internship, a Network-Based IDS was implemented using Snort, configured with custom detection rules and alerts to identify unusual traffic patterns and potential threats in real-time environments.

2.5 Integration of Cybersecurity Tools and Practices

Modern cybersecurity frameworks advocate a multi-layered defense approach, combining network monitoring, user awareness, secure software design, and automated intrusion detection. According to Kaur & Sharma (2022), a well-rounded cybersecurity strategy not only relies on tools but also on continuous training and incident response planning.

The internship experience reflected this integrated approach, allowing the intern to apply multiple concepts—such as sniffing for data capture, training for awareness, secure coding for prevention, and IDS for detection—collectively forming a comprehensive defence model.

3. Methodology

The methodology adopted during this internship focused on a structured, hands-on learning approach, combining theoretical concepts with practical implementation. The internship involved multiple cybersecurity tasks, each emphasizing a different domain — including network traffic analysis, phishing awareness, secure coding, and intrusion detection.

A systematic process was followed for each task, beginning with requirement analysis, followed by tool selection, design and implementation, testing, and documentation. This methodology ensured that every project aligned with professional cybersecurity standards and simulated real-world problem-solving environments.

3.1 Architecture Overview

The overall architecture of the internship projects was designed to reflect a multi-layered cybersecurity framework, incorporating data collection, analysis, detection, and response mechanisms.

The general workflow consisted of:

1. **Data Acquisition Layer:** This layer was responsible for collecting real-time data packets and network traffic. Tools such as Scapy were utilized for packet sniffing, enabling visualization of IP addresses, protocols, and payloads.
2. **Processing and Analysis Layer:** In this layer, data from the sniffer or intrusion detection tools was processed and analyzed to identify potential security events. The analysis involved examining traffic patterns, anomaly detection, and comparing results against known signatures or rules.
3. **Awareness and Training Layer:** To address the human aspect of cybersecurity, a Phishing Awareness Training module was developed. It acted as a proactive defensive layer aimed at reducing end-user susceptibility to social engineering attacks.
4. **Detection and Response Layer:** A Network Intrusion Detection System (NIDS) was configured using Snort and Suricata. This component monitored incoming and outgoing traffic in real-time and triggered alerts for suspicious activities.
5. **Secure Development Layer:** During the Secure Coding Review task, applications were examined for vulnerabilities. This layer represented the secure software development and auditing aspect of the architecture, aligning with OWASP guidelines.

The architecture can be visualized as a cybersecurity ecosystem, where each module complements the others — collectively contributing to prevention, detection, and response against cyber threats.

3.2 Hardware Components

Although most of the internship work was performed using virtual and software-based tools, certain basic hardware and system configurations were necessary to execute and test the projects efficiently. The following hardware setup was used during implementation:

Component	Specification	Purpose
Processor (CPU)	Intel Core i5 / Ryzen 5 or higher	To handle computation-heavy processes during packet capture and analysis
RAM	Minimum 8 GB	For smooth operation of virtualization and packet processing tools
Storage	256 GB SSD or higher	To store captured network data and configuration files
Network Adapter	Ethernet / Wi-Fi Adapter with Promiscuous Mode support	For capturing live network packets
Operating System Environment	Windows 10 / Ubuntu 22.04 LTS (Dual Boot or Virtual Machine)	To perform cross-platform testing and tool execution
Virtualization Tool	VMware Workstation / VirtualBox	For simulating network environments and IDS testing

Note: The hardware setup was designed to ensure scalability and performance optimization for tasks involving live traffic monitoring and multi-tool integration.

3.3 Software Tools

A range of **software tools and libraries** were employed to accomplish different cybersecurity tasks. The selection of tools was based on open-source availability, reliability, and compatibility with Python and Linux systems.

Tool / Technology	Description / Purpose	Application in Project
Python 3.x	High-level programming language used for automation and security scripting	Implemented for Network Sniffer and Secure Coding Review
Scapy	Python library for packet crafting and sniffing	Used to capture and analyze network packets
Socket Library	Standard Python library for low-level network communication	Enabled data transmission and connection testing
Wireshark	Network protocol analyzer	Used for cross-verifying packet data and inspecting network traffic
Snort	Open-source Network Intrusion Detection System	Implemented for monitoring and detecting suspicious traffic

Tool / Technology	Description / Purpose	Application in Project
Suricata	High-performance IDS/IPS engine	Used as an alternative to Snort for deep packet inspection
OWASP ZAP (Zed Attack Proxy)	Security scanner for web applications	Used during Secure Coding Review to detect vulnerabilities
GitHub	Version control and code hosting platform	Used for project submission and collaboration
Microsoft PowerPoint / Canva	Presentation tools	Used for creating Phishing Awareness Training material
VS Code / PyCharm	Integrated Development Environments (IDEs)	For code writing, debugging, and testing
Linux Terminal	Command-line interface for cybersecurity tools	For executing Snort and Suricata commands
VirtualBox	Virtualization platform	To simulate network environments for IDS testing

These tools collectively supported the research, implementation, and testing phases of the internship. Their integration ensured a balance between **practical learning** and **industry-standard practices** in cybersecurity.

4. Implementation

The implementation phase of the internship translated theoretical knowledge into practical applications by developing, configuring, and testing cybersecurity tools and solutions. Each task was approached methodically, focusing on identifying vulnerabilities, building countermeasures, and analyzing outcomes. This section explains the implementation of all four cybersecurity tasks performed during the internship — highlighting the technical process, tools used, and results obtained.

4.1 Task 1: Network Sniffer

Objective:

To design a Python-based tool that captures, analyzes, and displays real-time network packets to understand data flow and identify potential anomalies.

Implementation Process:

1. **Environment Setup:**

- Installed Python 3.10 and libraries such as scapy and socket.
- Verified network adapter's **promiscuous mode** capability for capturing all packets.

2. **Coding and Configuration:**

- Utilized the **Scapy** library to sniff packets from the network interface.
- Extracted and displayed packet details such as **Source IP, Destination IP, Protocol, and Payload Size**.
- Applied filters to isolate TCP, UDP, and ICMP packets for analysis.

3. **Testing:**

- Conducted tests using local Wi-Fi traffic.
- Observed DNS queries, HTTP requests, and ARP packets in real-time.

4. **Output:**

The tool successfully displayed real-time packet information, helping visualize network traffic and understand packet-level communication patterns.

Outcome:

The Network Sniffer enhanced the understanding of how data packets travel through networks and how attackers may exploit unencrypted traffic. It also served as a foundation for intrusion detection tasks.

4.2 Task 2: Phishing Awareness Training

Objective:

To create an interactive training module to educate users about phishing attacks, recognizing fraudulent emails, and preventing social engineering exploits.

Implementation Process:

1. Research and Content Preparation:

- Collected real-world phishing examples, including fake login pages and spoofed emails.
- Categorized phishing tactics (email scams, malicious links, fake websites).

2. Module Design:

- Created a PowerPoint and interactive PDF using **Canva** and **Microsoft PowerPoint**.
- Included educational sections on:
 - How phishing works
 - Common warning signs (e.g., suspicious links, urgent messages)
 - Verification methods (checking sender address, HTTPS links)
 - Preventive practices (2FA, email filters, awareness campaigns)
- Designed **interactive quizzes** and **visual flowcharts** to engage learners.

3. Implementation:

- Delivered the module as an online presentation and optional self-assessment form using **Google Forms**.
- Encouraged users to share results and feedback.

4. Outcome:

The training increased awareness about cyber deception tactics. Users demonstrated improved recognition of phishing attempts and better email hygiene practices.

4.3 Task 3: Secure Coding Review

Objective:

To identify and mitigate vulnerabilities within a sample web application through code auditing and static analysis techniques.

Implementation Process:

1. Environment Setup:

- Installed **OWASP ZAP**, **Visual Studio Code**, and **Bandit** (Python security linter).
- Selected a sample Python-based web application for testing.

2. Vulnerability Analysis:

- Performed **static code review** using Bandit to identify:
 - Hardcoded credentials
 - Insecure deserialization
 - Unvalidated user input
- Used **OWASP ZAP** for **dynamic scanning**, focusing on:
 - SQL Injection
 - Cross-Site Scripting (XSS)
 - Broken Authentication

3. Code Correction and Best Practices:

- Implemented mitigations like:
 - Input validation and parameterized queries
 - Secure password hashing using bcrypt
 - Removing hardcoded secrets and storing them in environment variables

4. Documentation:

- Created a **Secure Coding Checklist** based on OWASP Top 10 guidelines.

5. Outcome:

The Secure Coding Review improved the application's resilience against common exploits and reinforced understanding of software security principles.

4.4 Task 4: Network Intrusion Detection System (IDS)

Objective:

To set up and configure a **Network Intrusion Detection System** (NIDS) capable of monitoring network traffic and generating alerts for suspicious activities.

Implementation Process:

1. Environment Setup:

- Installed **Snort 3.1** on Ubuntu (using VirtualBox).
- Configured Snort in **Network Monitoring Mode** to analyze live traffic.
- Set up **Suricata** as an alternative IDS for comparison.

2. Rule Configuration:

- Created custom rules to detect ping floods, port scans, and suspicious IP activity.

3. Testing and Simulation:

- Simulated ping and port scan attacks using **Nmap**.
- Snort successfully generated alerts and logged them in `/var/log/snort/alert`.
- Verified alerts using **Base (Basic Analysis and Security Engine)** dashboard.

4. Output Visualization:

- Configured a local dashboard to visualize detected threats and network events.

5. Outcome:

The IDS effectively detected anomalies in network traffic and generated actionable alerts, demonstrating its potential in real-world monitoring environments

5. Results and Discussion

The results obtained from the internship demonstrate a comprehensive understanding of cybersecurity principles and their real-world applications. Each task provided unique insights into network behaviour, system vulnerabilities, and preventive defense mechanisms. This section presents the outcomes, performance analysis, and interpretations derived from the four major tasks—**Network Sniffer**, **Phishing Awareness Training**, **Secure Coding Review**, and **Network Intrusion Detection System (IDS)**—along with overall learning observations.

5.1 Task 1: Network Sniffer

Results:

The **Network Sniffer** successfully captured live packets and displayed details such as **Source IP**, **Destination IP**, **Protocol Type**, and **Payload Size** in real time. The tool accurately differentiated between TCP, UDP, ICMP, and ARP packets. It also logged packet data for further offline analysis.

Analysis:

- The use of **Scapy** allowed packet-level visibility, providing a clearer understanding of how data moves through the network.
- Testing on local and simulated environments revealed how frequent ARP and DNS requests occur even in idle network conditions.
- Observations indicated that unencrypted traffic (HTTP) could easily expose sensitive data such as login credentials and cookies, highlighting the necessity of using secure protocols like HTTPS.

Discussion:

The project demonstrated how **network sniffing** can be both a diagnostic tool and a potential attack vector if misused. While network administrators rely on sniffers for traffic analysis, malicious actors can exploit them to intercept data. Thus, implementing encryption and network segmentation is crucial to mitigate such risks.

5.2 Task 2: Phishing Awareness Training

Results:

The **Phishing Awareness Module** was successfully designed and presented as an interactive training session. It included educational slides, real-world phishing examples, quizzes, and preventive tips. Post-training feedback revealed that participants showed improved awareness and could identify phishing emails with greater accuracy.

Analysis:

- Participants learned to inspect sender domains, hover over hyperlinks before clicking, and verify SSL certificates on websites.
- The module demonstrated that awareness training can significantly reduce human error—a leading cause of data breaches.
- Incorporating real-world phishing samples made the training relatable and more impactful.

Discussion:

The exercise underscored that **user education** is as vital as technical defenses. Phishing remains one of the most effective attack methods because it exploits human trust. Continuous awareness programs, simulated phishing campaigns, and feedback loops are essential for maintaining a vigilant user base.

5.3 Task 3: Secure Coding Review

Results:

The **Secure Coding Review** identified multiple vulnerabilities in the sample web application:

- Hardcoded credentials
- Unvalidated user inputs (potential SQL Injection risk)
- Weak password storage (plain text)
- Missing error-handling mechanisms

These issues were successfully remediated through **input validation, parameterized queries, hashed credentials, and secure API calls.**

Analysis:

- Static analysis tools like **Bandit** quickly pinpointed insecure coding patterns.
- **OWASP ZAP** confirmed the absence of SQLi and XSS vulnerabilities after remediation.

- The final codebase adhered to OWASP Top 10 security recommendations, showcasing measurable improvement in software integrity.

Discussion:

This task emphasized that **security must be integrated into the software development lifecycle (SDLC)**. Regular code reviews and automated scans are necessary to detect vulnerabilities early. Secure coding is not just a corrective practice but a preventive culture that every developer must adopt.

5.4 Task 4: Network Intrusion Detection System (IDS)

Results:

The **Intrusion Detection System (IDS)** implemented using **Snort** and **Suricata** successfully monitored network traffic and generated alerts upon detecting suspicious activities such as ICMP flooding, port scanning, and unauthorized HTTP requests.

Analysis:

- The IDS efficiently distinguished between normal and anomalous traffic using predefined rule sets.
- During testing, the system produced low false positives, demonstrating the accuracy of the configured detection rules.
- Visualization dashboards enhanced situational awareness by graphically representing attack attempts and traffic anomalies.

Discussion:

The IDS implementation showcased how **proactive monitoring and early detection** can prevent network breaches. Tools like Snort provide a cost-effective and customizable solution for organizations to enhance their network defense. The project also highlighted the need for integrating IDS with automated response systems for faster containment of threats.

5.5 Comparative Discussion and Key Findings

Task	Primary Focus	Key Result	Impact
Network Sniffer	Packet analysis	Successfully captured and analyzed live network packets	Improved understanding of network protocols
Phishing Awareness	User education	Enhanced user ability to identify phishing attempts	Strengthened human-centric security
Secure Coding Review	Application-level security	Identified and remediated major coding vulnerabilities	Improved software resilience
Intrusion Detection System	Network monitoring	Detected and logged real-time network threats	Strengthened threat detection capabilities

Overall Key Findings:

- 1. Cybersecurity requires both **technical and human-focused defences**.
- 2. **Automation and tool-based detection** improve efficiency but still rely on accurate configurations.
- 3. Continuous **security training, code auditing, and monitoring** are crucial for resilience.
- 4. Open-source tools like **Scapy, Snort, and OWASP ZAP** provide powerful, cost-effective solutions for learning and implementation.

5.6 Overall Discussion

The results of the internship reaffirm that cybersecurity is not a single-step process but a **multi-layered discipline** involving continuous assessment, awareness, and response. The combination of technical tasks (sniffer, IDS, secure coding) and non-technical elements (phishing awareness) provided a holistic understanding of modern security ecosystems.

The practical exposure to real-time packet capturing, intrusion detection configuration, and vulnerability analysis bridged the gap between classroom theory and industry requirements. Moreover, the iterative process of testing and improvement reflected real-world cybersecurity workflows—detect, analyze, respond, and document.

6. Conclusion and Future Scope

6.1 Conclusion

The **Cyber Security Internship** at **CodeAlpha** provided invaluable hands-on exposure to real-world security challenges, practical implementations, and industry-standard tools. The internship was not limited to theoretical understanding but emphasized **experiential learning through project-based execution** of critical cybersecurity concepts such as **network traffic analysis, phishing awareness, secure coding, and intrusion detection systems**.

Throughout the internship, the systematic approach of identifying vulnerabilities, designing solutions, and validating results significantly enhanced both technical and analytical proficiency. The tasks performed demonstrated a complete lifecycle of cybersecurity operations—starting from **data collection (sniffing)** to **threat detection (IDS)**, and from **prevention through secure coding** to **education via phishing awareness**.

Key takeaways from the internship include:

- Understanding the **importance of proactive monitoring** through packet analysis and IDS tools.
- Recognizing the **crucial role of end-user education** in preventing social engineering attacks.
- Gaining proficiency in **secure software development practices** aligned with OWASP standards.
- Developing the ability to **configure and test security mechanisms** in real-time environments.

The internship also nurtured essential professional attributes such as **problem-solving, documentation, teamwork, and self-learning**, which are vital for any cybersecurity practitioner. It served as a bridge between academic knowledge and professional application, providing a comprehensive perspective on modern cybersecurity ecosystems.

In conclusion, the internship has not only deepened technical understanding but also instilled a sense of responsibility towards building **secure digital infrastructures**. The experience has laid a solid foundation for pursuing advanced studies and a professional career in the field of cybersecurity.

6.2 Future Scope

Cybersecurity is an ever-evolving discipline that constantly adapts to emerging threats, technologies, and attack methodologies. The tasks and learning outcomes from this internship open multiple avenues for further exploration, research, and development in the cybersecurity domain.

Future Scope and Recommendations:

1. **Integration of Artificial Intelligence in Cyber Defence:** Implementing AI and Machine Learning models can improve the accuracy of intrusion detection systems by automatically classifying and predicting threat patterns.
2. **Development of Automated Phishing Detection Tools:** Building browser extensions or email filters using Natural Language Processing (NLP) can help identify phishing emails in real time and alert users proactively.
3. **Advanced Secure Coding Frameworks:** Future work can focus on integrating static and dynamic analysis tools into CI/CD pipelines to detect vulnerabilities automatically during software builds.
4. **Enhanced Network Monitoring with Visualization:** Developing dashboards for real-time visualization of network anomalies and packet flow analytics can make monitoring more efficient and intuitive.
5. **Cloud Security and Zero Trust Implementation:** Expanding the scope to include **cloud-based intrusion detection** and **Zero Trust architectures** can address security challenges in distributed systems.
6. **Penetration Testing and Red Team Exercises:** Conducting simulated attacks can strengthen defensive strategies and provide deeper insights into attacker methodologies.
7. **User-Centric Security Education Platforms:** Extending the phishing awareness model into an e-learning platform can help train employees and students on cybersecurity best practices across industries

References

1. Bhattacharya, P., & Islam, R. (2020). *A Survey on Network Packet Analysis and Monitoring Tools*. *Journal of Network Security and Applications*, 12(3), 45–58.
2. Denning, D. E. (1987). *An Intrusion-Detection Model*. *IEEE Transactions on Software Engineering*, 13(2), 222–232. <https://doi.org/10.1109/TSE.1987.232894>
3. García-Teodoro, P., Díaz-Verdejo, J., Maciá-Fernández, G., & Vázquez, E. (2009). *Anomaly-based Network Intrusion Detection: Techniques, Systems and Challenges*. *Computers & Security*, 28(1–2), 18–28. <https://doi.org/10.1016/j.cose.2008.08.003>
4. Gupta, S., & Singhal, N. (2021). *Effectiveness of Phishing Awareness Training Programs*. *Information Security Journal: A Global Perspective*, 30(1), 15–29. <https://doi.org/10.1080/19393555.2021.1877619>
5. Jagatic, T. N., Johnson, N. A., Jakobsson, M., & Menczer, F. (2007). *Social Phishing*. *Communications of the ACM*, 50(10), 94–100. <https://doi.org/10.1145/1290958.1290968>

